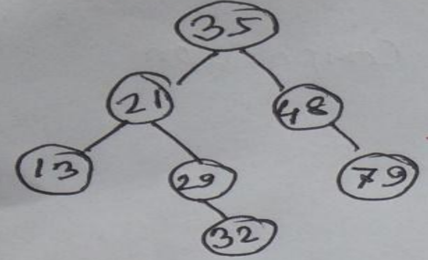
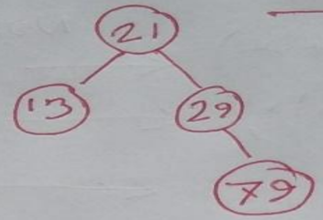
Balanced Tree and Heap

Balanced Tree (AVL)



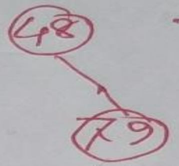
Left sub-tree



$height_1 = 2 \ (h_1)$

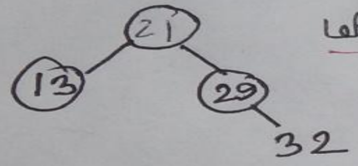
since $|h_1 - h_2| <= 1$
So. This is balanced tree

Right subtree



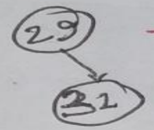
$height_2 = 1 \ (h_2)$

Check in ~~left~~ left-subtree — Left of left subtree



Left of left subtree → 13] $h_1 = 0$

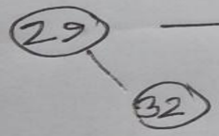
~~Left~~ Right of left subtree →



] $h_2 = 1$

since $|h_1 - h_2| <= 1$
so. This is a balanced tree

Check in right subtree of left subtree



left subtree → empty] $h_1 = -1$

Right subtree → 32 $h_2 = 0$

since $|h_1 - h_2| <= 1$
so tree is balanced

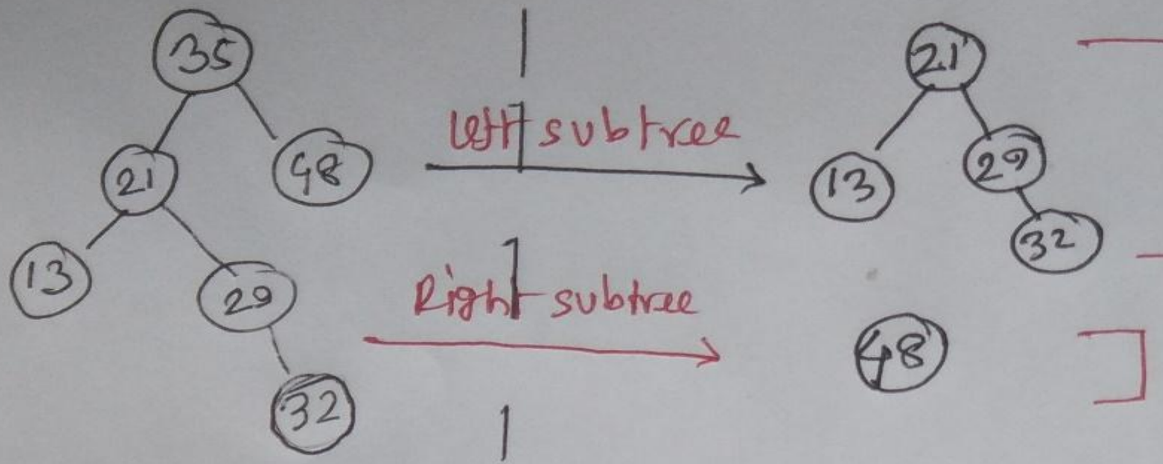
32

Left subtree → empty] $h_1 = -1$

Right subtree → empty] $h_2 = -1$

since $|h_1 - h_2| <= 1$
so tree is balanced

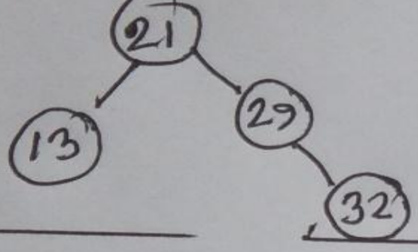
Balanced Tree (AVL)



$h_1 = 2$

$h_2 = 0$

since $|h_1 - h_2| > 1$
so The tree is not balanced

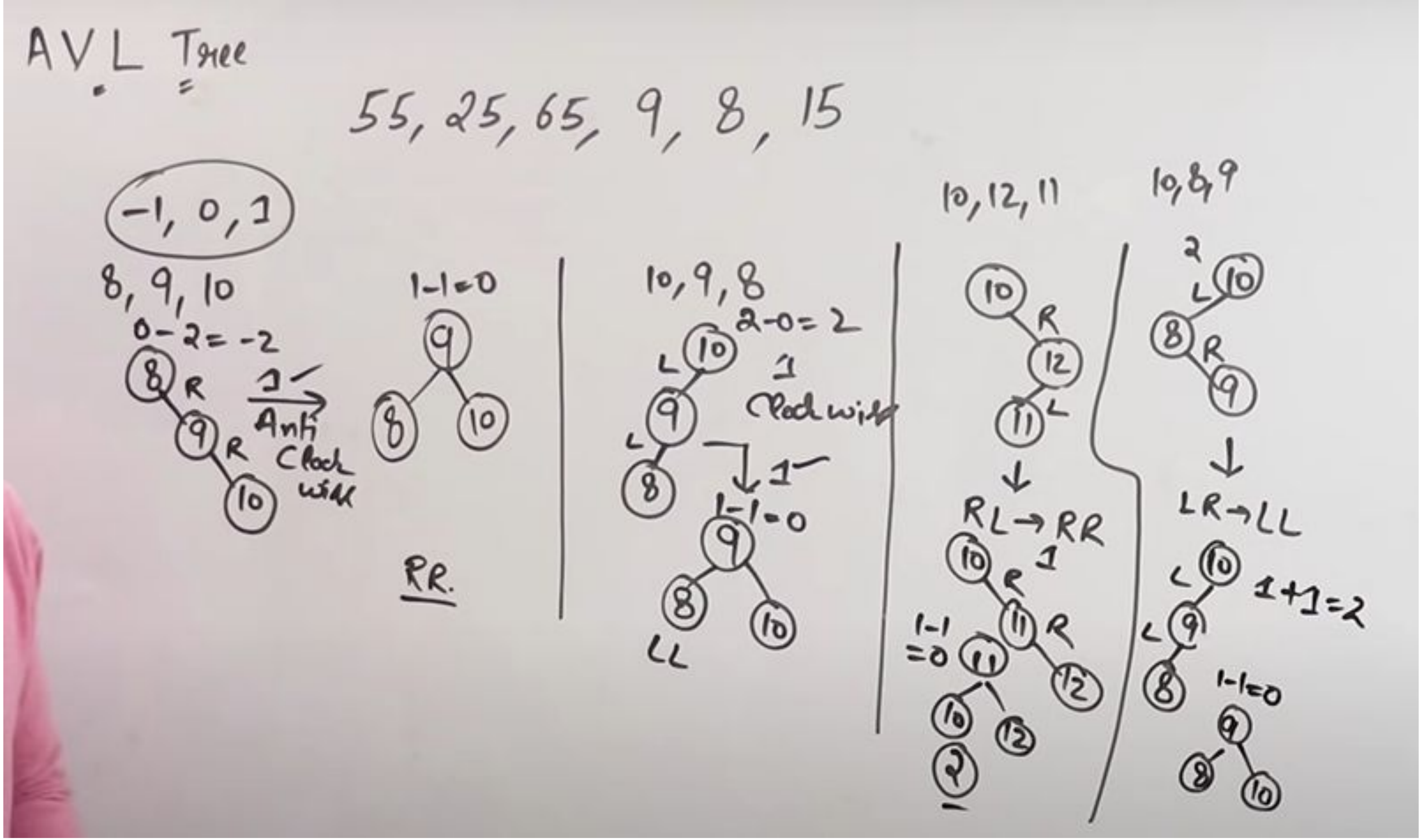


⇒ This tree is a balanced tree.

~~Since~~,

Since The tree created an unbalanced situation at the begining. so The tree is Unbalanced.

Balanced Tree (AVL)



Balanced Tree (AVL)

How many rotations are required during construction of an AVL tree if the following elements are added in order given?

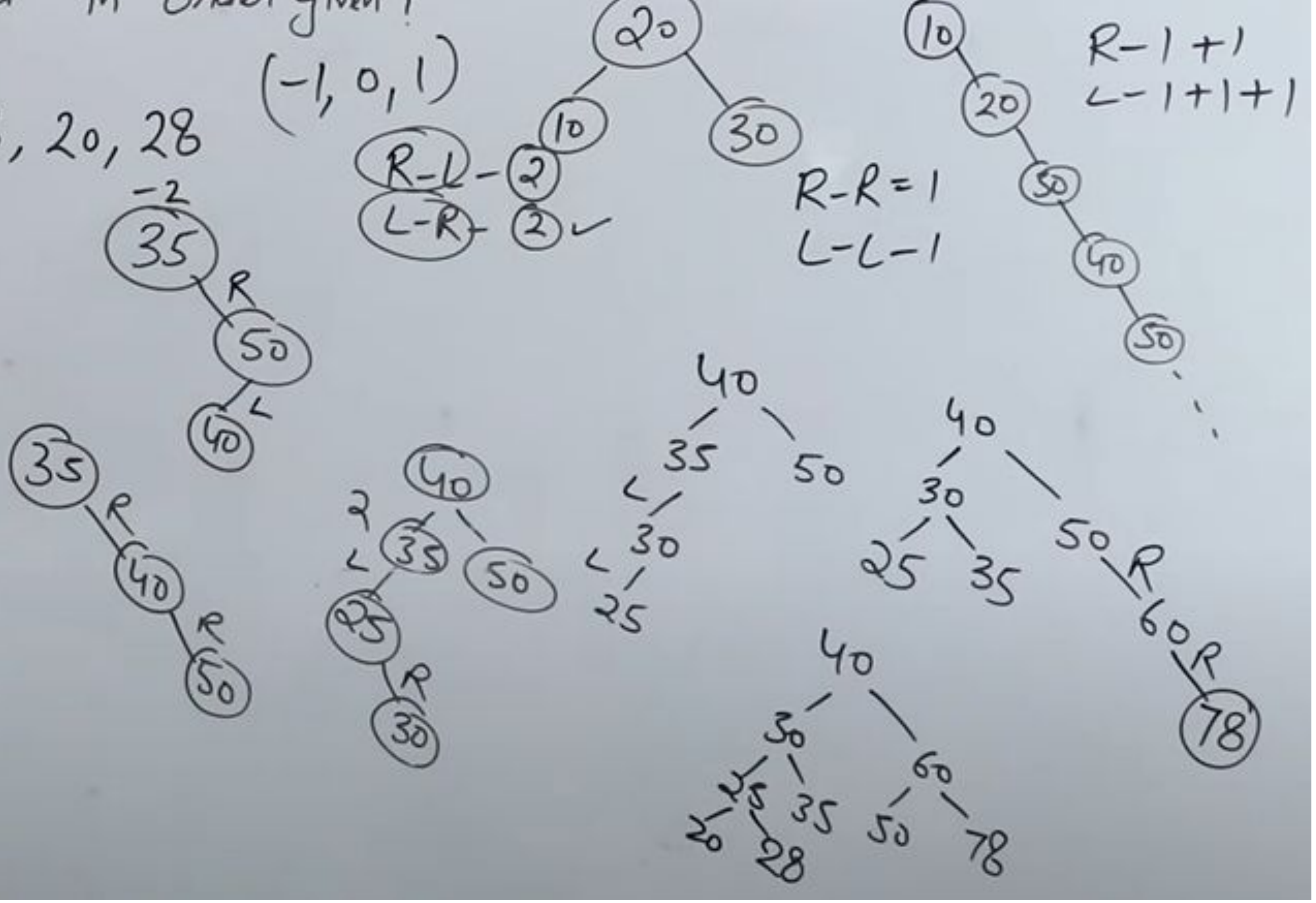
35, 50, 40, 25, 30, 60, 78, 20, 28

a) 2 Left, 3 Right

b) 3 left, 2 Right

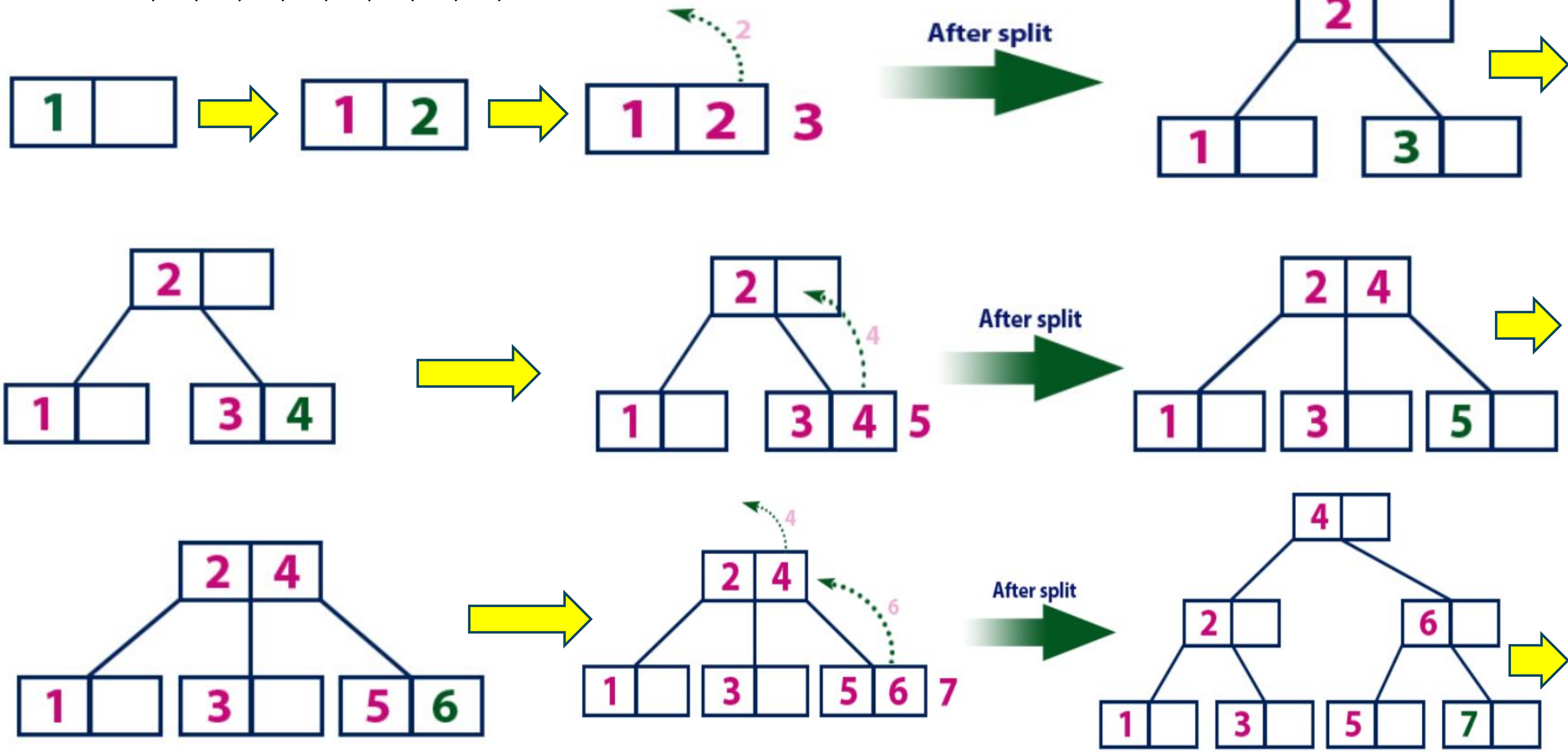
c) 2 left, 2 Right

d) 2 left, 1 Right

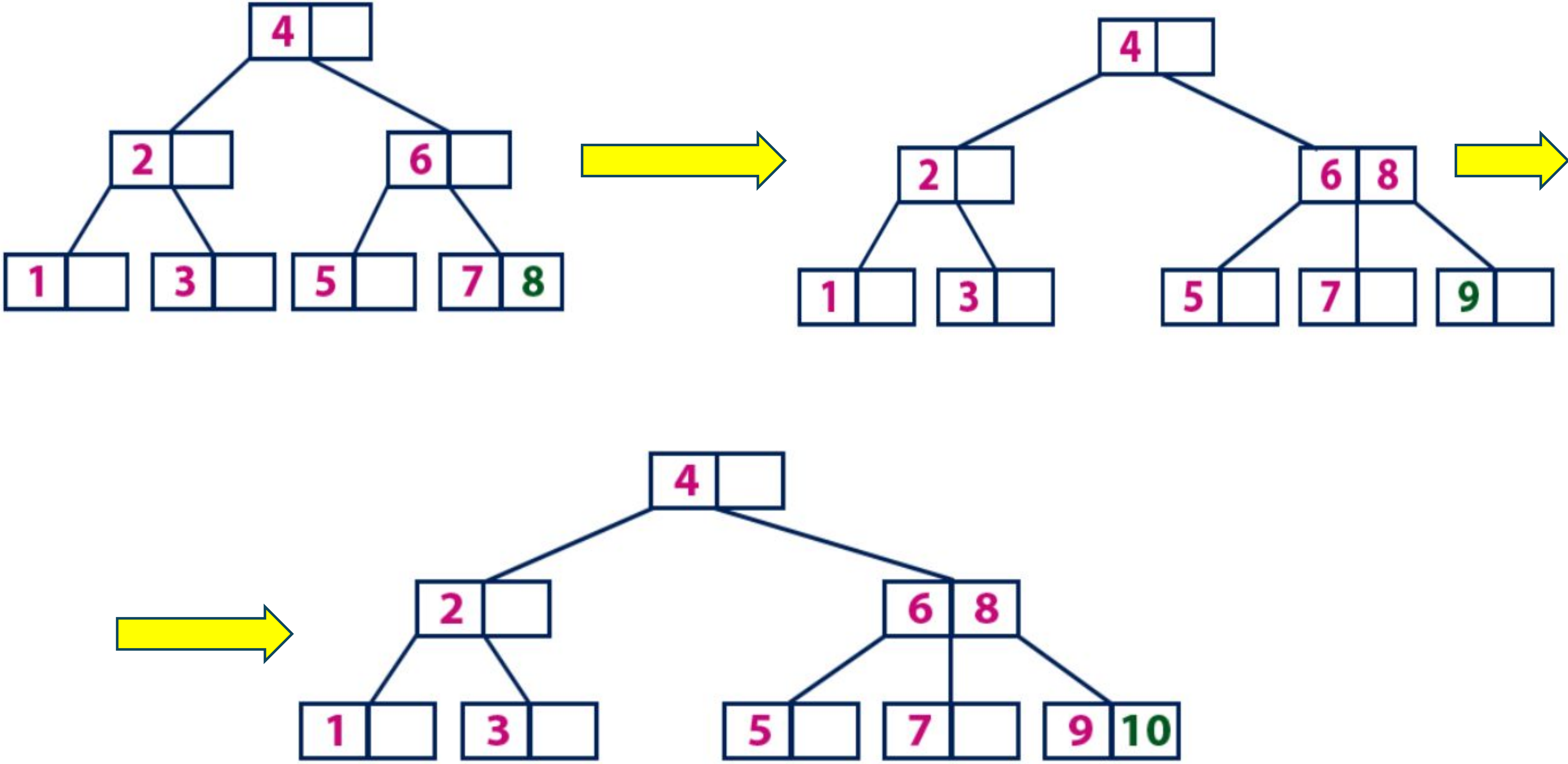


B-Tree structure

List: 1,2,3,4,5,6,7,8,9,10

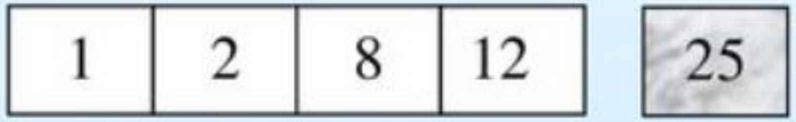


B-Tree structure



B-Tree structure

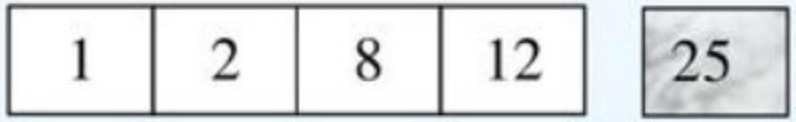
- Suppose we start with an empty B-tree and keys arrive in the following order:1 12 8 2 25 5 14 28 17 7 52 16 48 68 3 26 29 53 55 45
- We want to construct a B-tree of order 5
- The first four items go into the root:

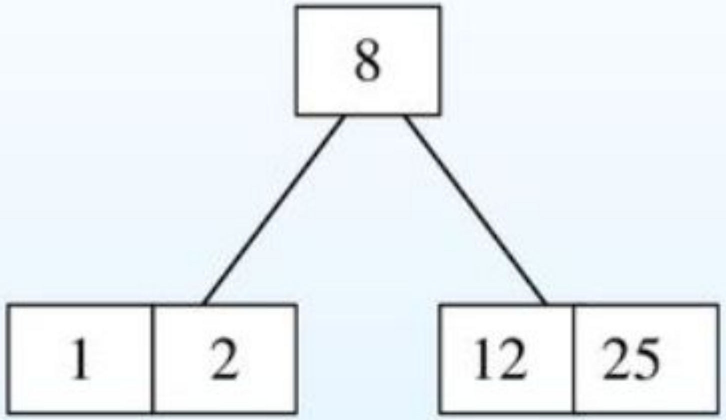


- To put the fifth item in the root would violate condition 5
- Therefore, when 25 arrives, pick the middle key to make a new root

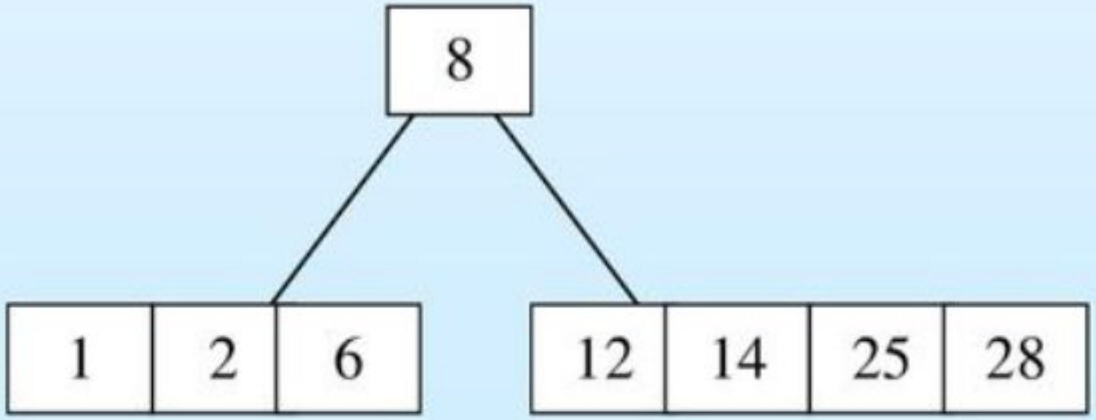
https://slideplayer.com/slide/16807097/#.ZBLt8pKsMfA.whatsapp

B-Tree structure

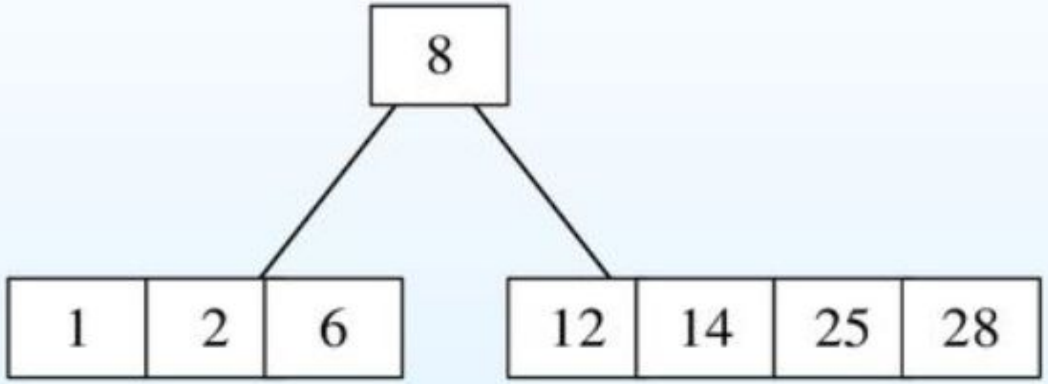




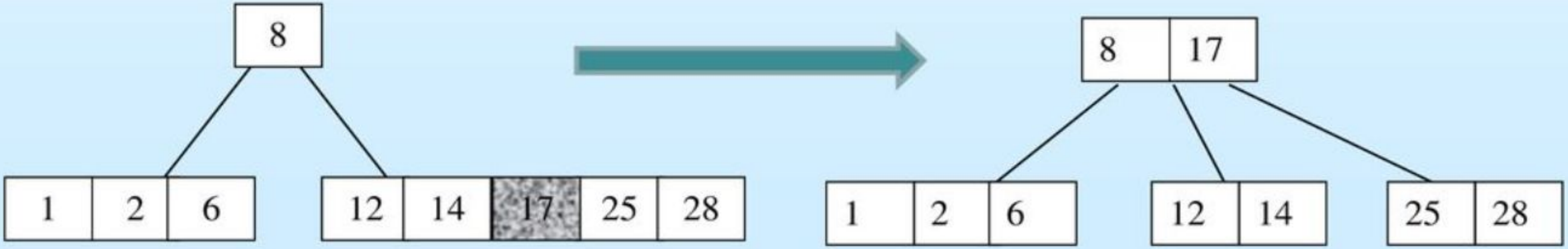
6, 14, 28 get added to the leaf nodes:



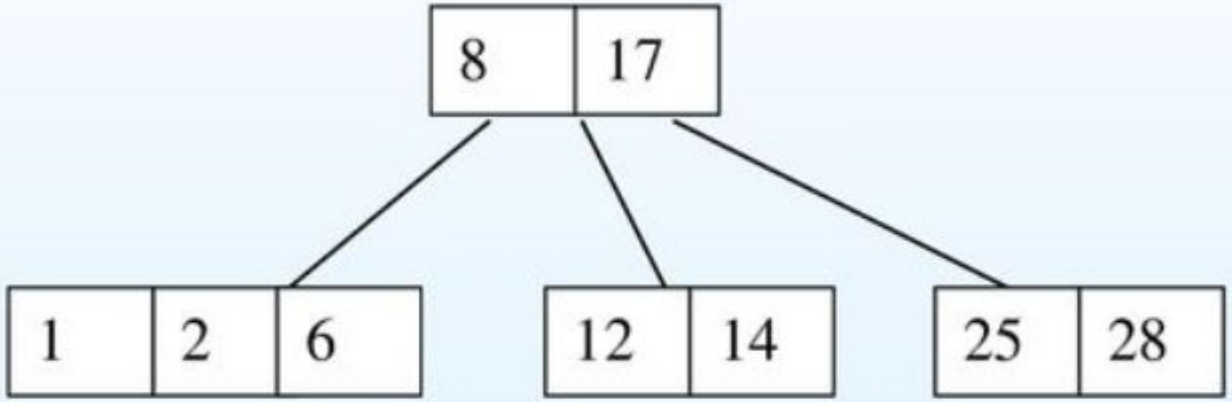
B-Tree structure



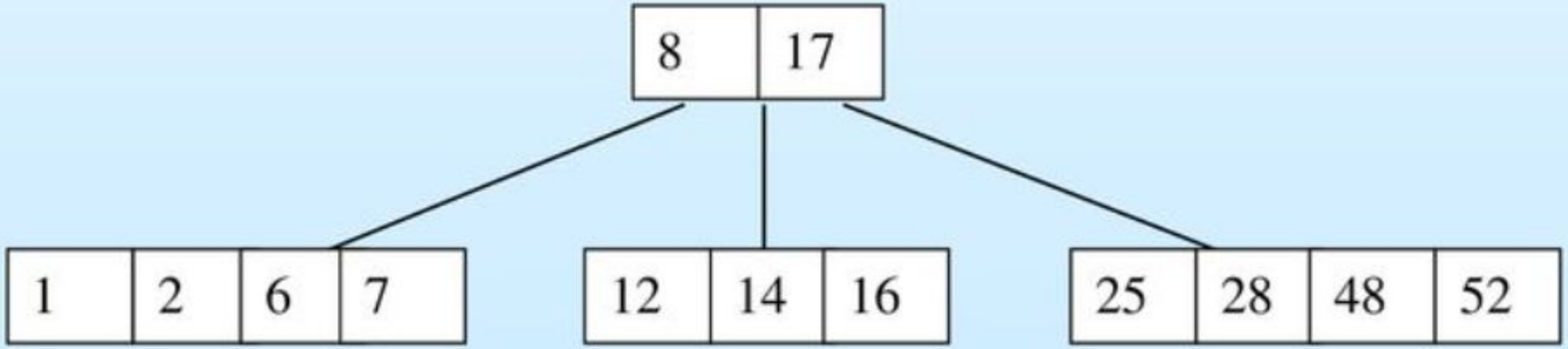
Adding 17 to the right leaf node would over-fill it, so we take the middle key, promote it (to the root) and split the leaf



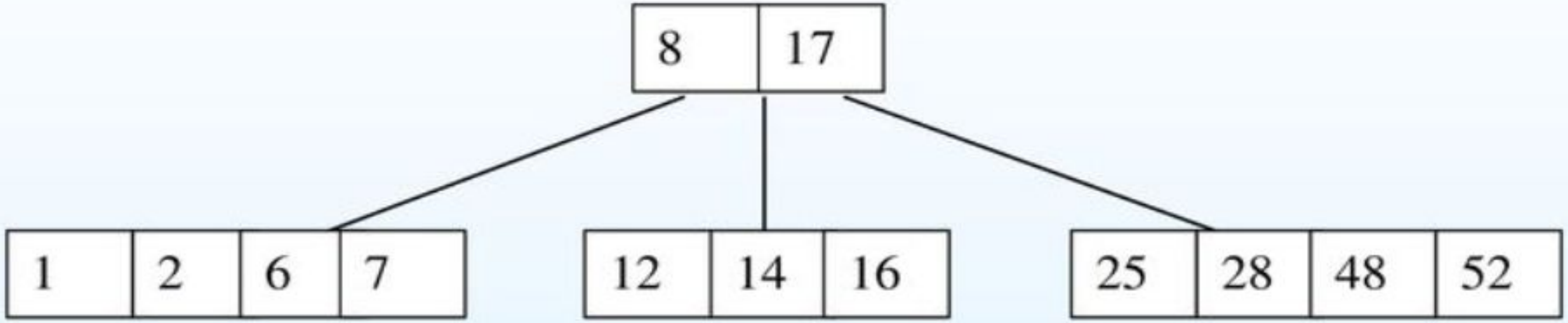
B-Tree structure



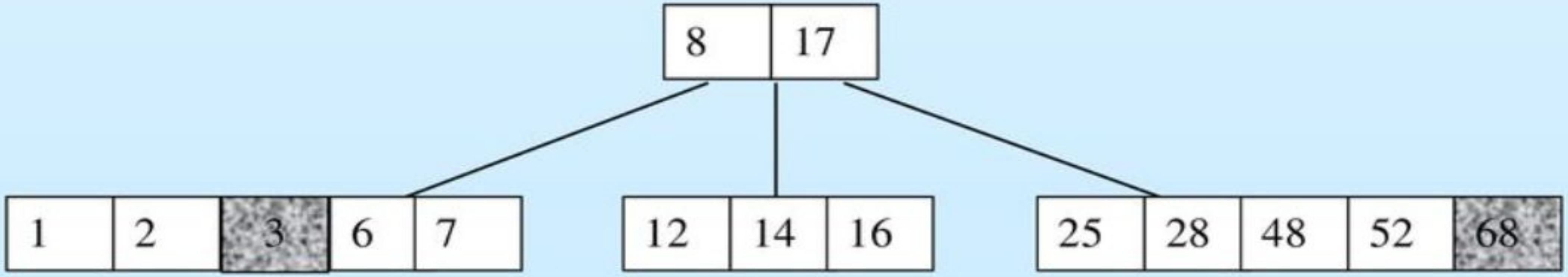
7, 52, 16, 48 get added to the leaf nodes



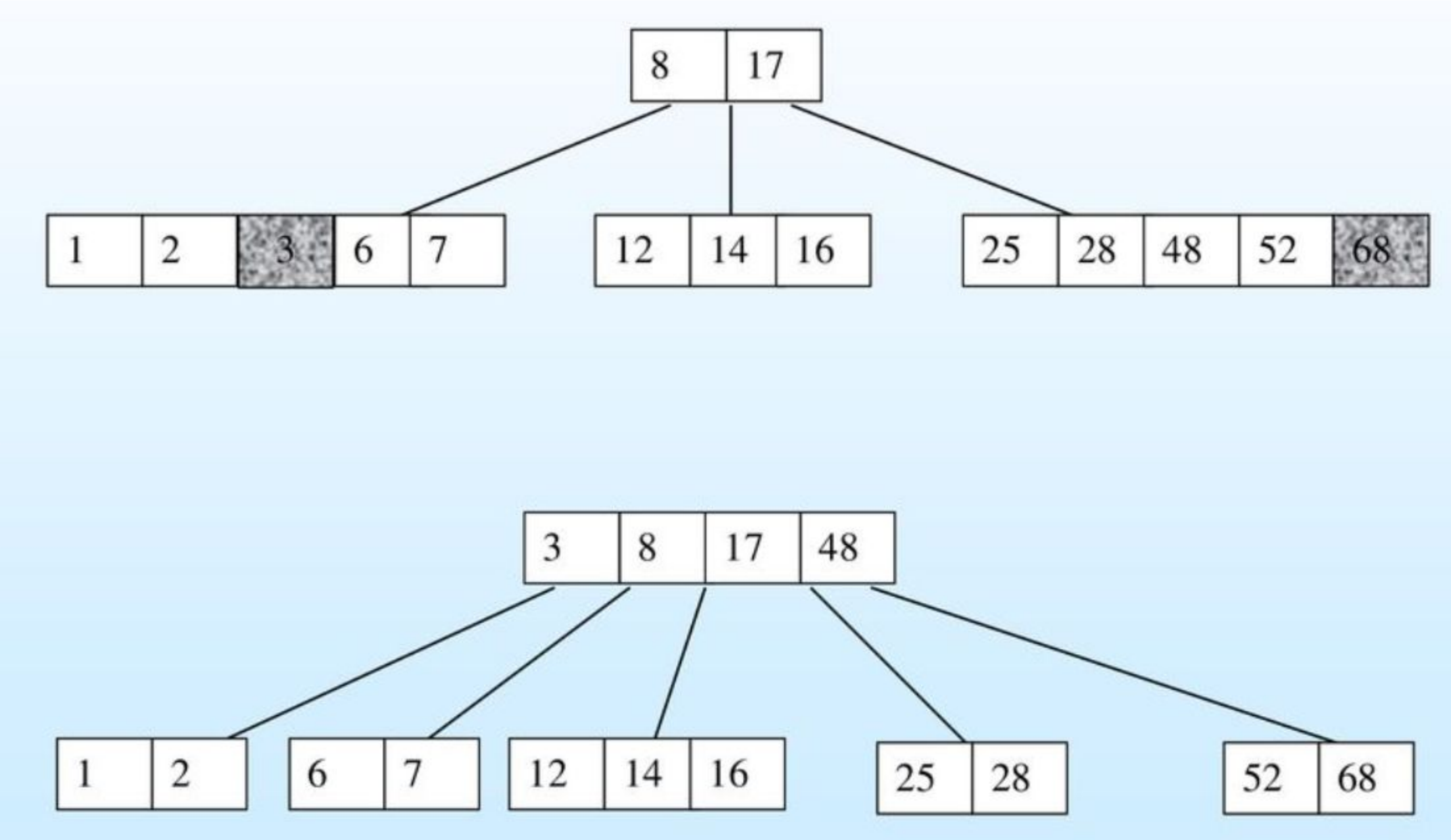
B-Tree structure



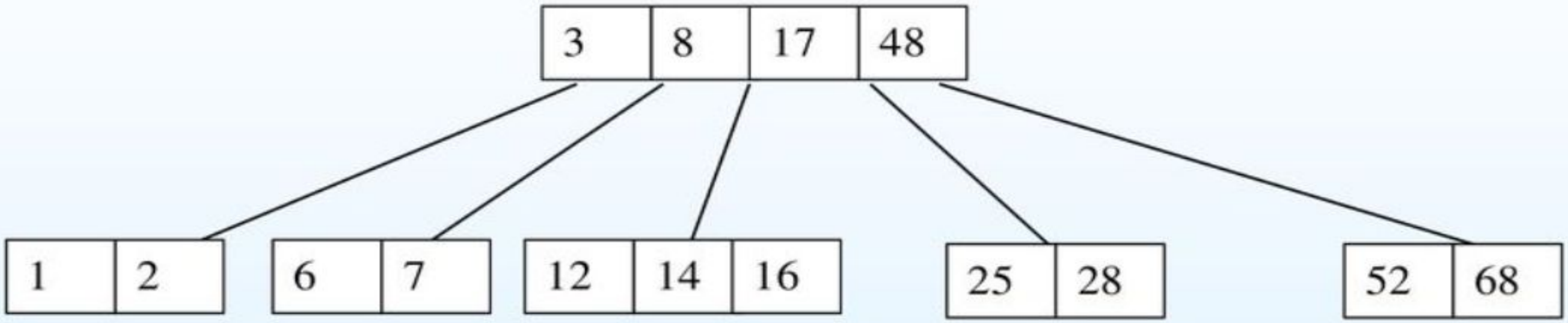
Adding 68 causes us to split the right most leaf, promoting 48 to the root, and adding 3 causes us to split the left most leaf, promoting 3 to the root



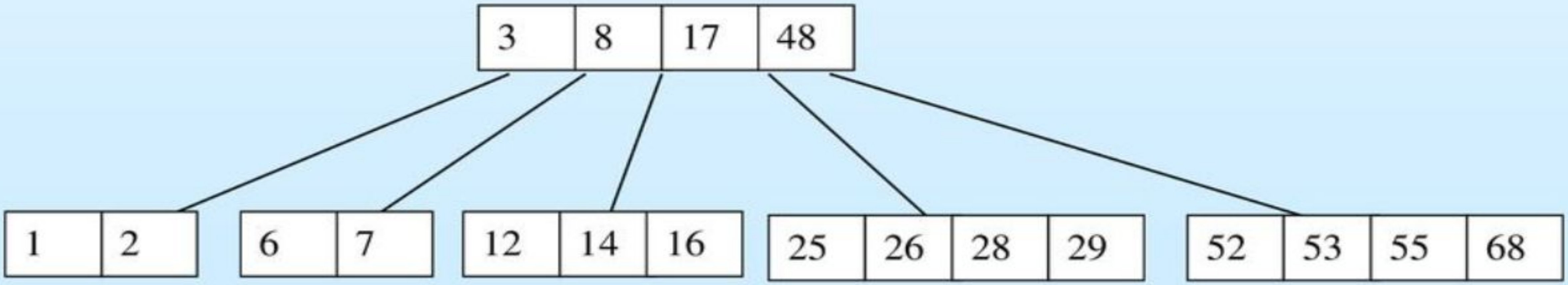
B-Tree structure



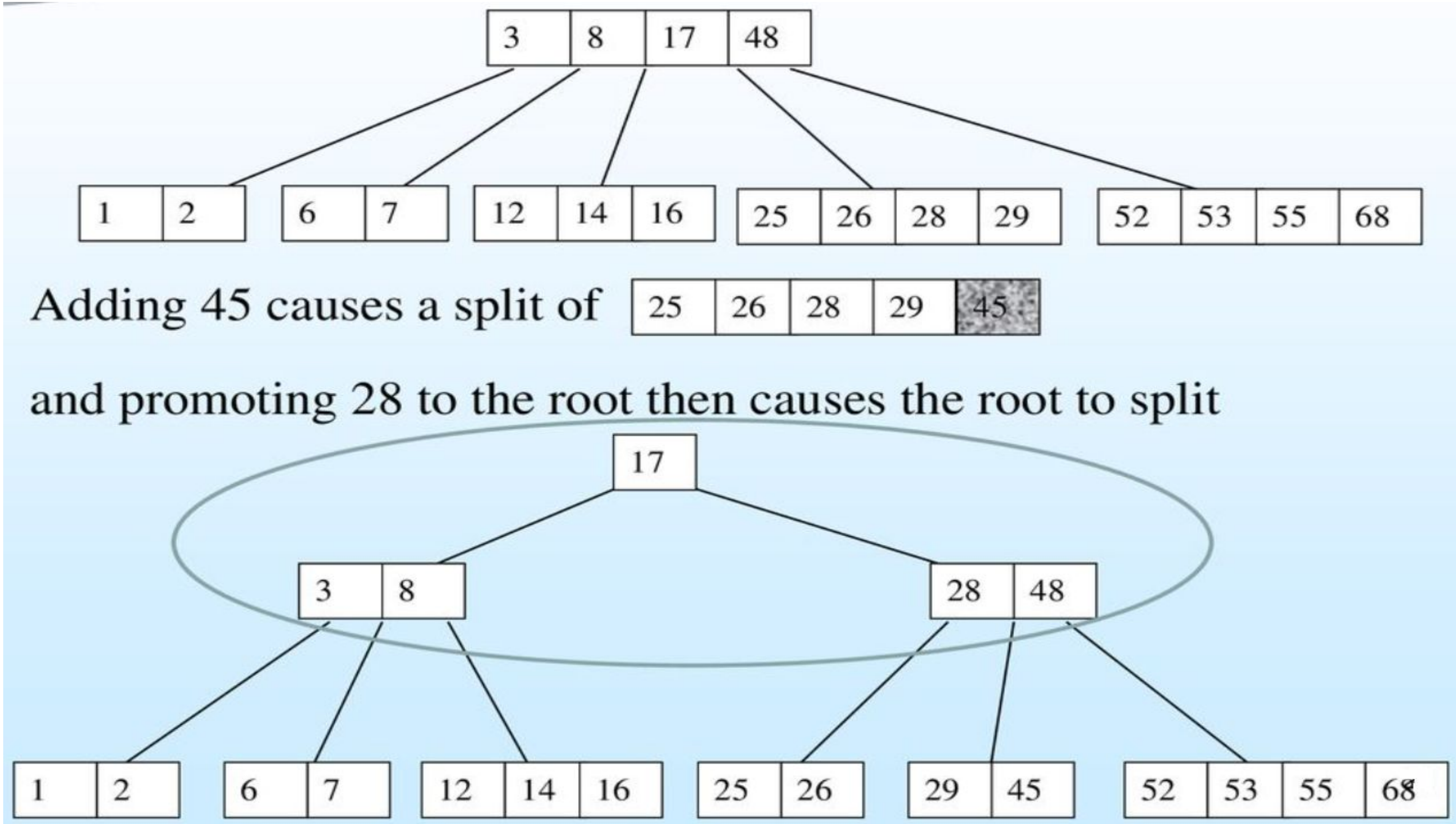
B-Tree structure



Adding 26, 29, 53, 55: go into the leaves



B-Tree structure



Heap

Thank You